

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: ПОИСК ПОДСТРОКИ В СТРОКЕ.

Студент гр. 4343

Маркашов Г.Д.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель Работы.

Изучение алгоритмов нахождения подстроки в строке и нахождение префикс функции строки.

Задание 1.

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

Первая строка – P

Вторая строка – T

Выход:

индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Входные данные

ab

abab

Выходные данные

0,2

Задание 2.

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

Первая строка – P

Вторая строка – T

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Входные данные

defabc

abcdef

Выходные данные 3

Теоретический минимум

Алгоритм КМП предназначен для эффективного поиска всех вхождений строки-шаблона P в строку-текст T . Его основная идея заключается в использовании префикс-функции, которая позволяет избежать повторной проверки уже совпавших символов при возникновении несовпадения

Временная и пространственная сложность алгоритма составляет $O(P + T)$.

Префикс-функция

Префикс-функция $pi(i)$ для строки s определяется как длина наибольшего собственного префикса строки $s[0...i]$, который одновременно является её суффиксом.

Алгоритм вычисления префикс-функции:

- Инициализируем массив pi нулями
- Для каждого i от 1 до $n - 1$:
 - о Берём значение $j = pi[i - 1]$ (длина предыдущего совпадения)
 - о Пока $j > 0$ и $s[i] \neq s[j]$, уменьшаем j до $pi[j - 1]$
 - о Если $s[i] == s[j]$, увеличиваем j на 1
 - о Записываем $pi[i] = j$

Поиск подстроки

После вычисления префикс-функции для шаблона P выполняется проход по тексту T :

- Поддерживается переменная j — текущая длина совпадения
- При совпадении символов j увеличивается

- При несовпадении j уменьшается с использованием префиксфункции
- Когда j достигает длины шаблона, фиксируется вхождение

Особенность выполнения задания 2: Так как у нас 2 строки и одно из которых циклическое смещение в другой строке, то достаточно строку для проверки с контекстировать с самой собой и использовать уже имеющийся алгоритм.

Тестирование

Задание 1.

Входные данные	Выходные данные
ab abab	0,2
aba abacaba	0,4
a banana	1,3,5
abab abababab	0,2,4

Задание 2.

Входные данные	Выходные данные
defabc abcdef	3
cab abc	1
aaa aaa	0
zc uio	-1

Приложения

Приложение А

kmp.py:

```
def get_prefix_func(s: str) -> list[int]:
    n = len(s)
    pi = [0] * n

    print(f"Запускаем префикс-функцию для шаблона: {s}")
    print("Всегда pi[0] = 0")

    for i in range(1, n):
        print(f"\nИтерация для индекса i = {i}")
        print(f"Рассматриваем подстроку: \"{s[:i+1]}\"")

        j = pi[i-1]
        print(f"Берем значение pi от прошлого элемента: j = pi[{i-1}]")

        while j > 0 and s[i] != s[j]:
            print(f"Откат: s[{i}]('{s[i]}') != s[{j}]('{s[j]}'),")
            print(f"новый j = pi[{j-1}] = {pi[j-1]}")
            j = pi[j-1]

        if s[j] == s[i]:
            j += 1
            print(f"СОВПАДЕНИЕ: s[{i}] == s[{j-1}], увеличиваем j на")
        else:
            print(f"НЕТ СОВПАДЕНИЯ: s[{i}] != s[{j}], j остается")

        pi[i] = j
        print(f"Результат: pi[{i}] = {j}")
        print(f"Текущий массив pi: {pi[:i+1]}")

    print(f"\nИтоговая префикс-функция: {pi}\n" + "="*40)
    return pi

def kmp(text: str, pattern: str) -> list[int]:
    nt = len(text)
    np = len(pattern)
    if np == 0: return []

    pi = get_prefix_func(pattern)
    print(f"\nЗапускаем алгоритм КМП")
    print(f"Текст: {text}\nШаблон: {pattern}")
```

```

res = []
j = 0

for i in range(nt):
    print(f"\nШаг i={i} (символ текста '{text[i]}')")

    while j > 0 and text[i] != pattern[j]:
        print(f"    Элементы не совпали: text[{i}] != pattern[{j}],
откат j = pi[{j-1}] = {pi[j-1]}")
        j = pi[j-1]

    if text[i] == pattern[j]:
        print(f"    Элементы совпали: text[{i}] == pattern[{j}], j =
{j+1}")
        j += 1

    if j == np:
        print(f"НАЙДЕНО ВХОЖДЕНИЕ по индексу {i - np + 1}")
        res.append(i - np + 1)
        print(f"    Откатываем j для поиска перекрытий: j =
pi[{j-1}] = {pi[j-1]}")
        j = pi[j-1]
    else:
        print(f"    j = 0 и совпадений нет, просто идем к следующему
i")

return res

if __name__ == "__main__":
    pattern = input()
    text = input()

    result = kmp(text, pattern)

    print("\nФИНАЛЬНЫЙ ОТВЕТ:")
    if result:
        print(",".join(map(str, result)))
    else:
        print(-1)

```

5_2.py:

```

def get_prefix_func(s: str) -> list[int]:
    n = len(s)
    pi = [0] * n

    print(f"Запускаем префикс-функцию для шаблона: {s}")
    print("Всегда pi[0] = 0")

```

```

for i in range(1, n):
    print(f"\nИтерация для индекса i = {i}")
    print(f"Рассматриваем подстроку: \"{s[:i+1]}\"")

    j = pi[i-1]
    print(f"Берем значение pi от прошлого элемента: j = pi[{i-1}]
= {j}")

    while j > 0 and s[i] != s[j]:
        print(f"    Откат: s[{i}]{s[i]} != s[{j}]{s[j]}"),
        НОВЫЙ j = pi[j-1] = {pi[j-1]}
        j = pi[j-1]

    if s[j] == s[i]:
        j += 1
        print(f"    СОВПАДЕНИЕ: s[{i}] == s[{j-1}], увеличиваем j на
1: j = {j}")
    else:
        print(f"    НЕТ СОВПАДЕНИЯ: s[{i}] != s[{j}], j остается
{j}")

    pi[i] = j
    print(f"    Результат: pi[{i}] = {j}")
    print(f"    Текущий массив pi: {pi[:i+1]}")

print(f"\nИтоговая префикс-функция: {pi}\n" + "="*40)
return pi

def kmp(text: str, pattern: str) -> list[int]:
    nt = len(text)
    np = len(pattern)
    if np == 0: return []

    pi = get_prefix_func(pattern)
    print(f"\nЗапускаем алгоритм КМП")
    print(f"Текст: {text}\nШаблон: {pattern}")

    res = []
    j = 0

    for i in range(nt):
        print(f"\nШаг i={i} (символ текста '{text[i]}')")

        while j > 0 and text[i] != pattern[j]:
            print(f"    Элементы не совпали: text[{i}] != pattern[{j}],
откат j = pi[j-1] = {pi[j-1]}")
            j = pi[j-1]

        if text[i] == pattern[j]:
            print(f"    Элементы совпали: text[{i}] == pattern[{j}], j =
{j+1}")
            j += 1

        if j == np:
            print(f"НАЙДЕНО ВХОЖДЕНИЕ по индексу {i - np + 1}")
            res.append(i - np + 1)

```



```

        print(f" Откатываем j для поиска перекрытий: j =
pi[{j-1}] = {pi[j-1]}")
        j = pi[j-1]
    else:
        print(f" j = 0 и совпадений нет, просто идем к следующему
i")

    return res

if __name__ == "__main__":
    A = input()
    B = input()

    print("\nФИНАЛЬНЫЙ ОТВЕТ:")
    if len(B) != len(A):
        print(-1)
    else:
        result = kmp(A*2, B)
        if result:
            print(result[0])
        else:
            print(-1)

```